

Cache and Virtual Memories

Prof. Claudio Passerone

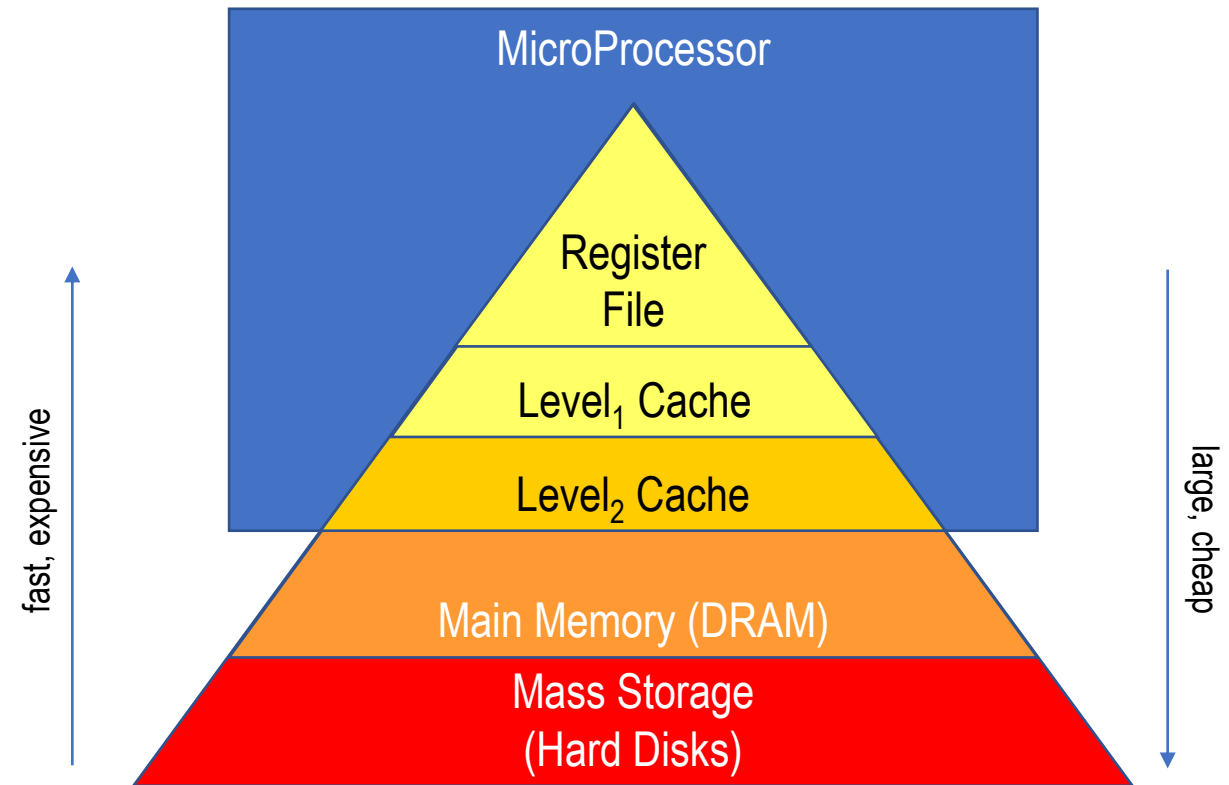
(Electronics for Embedded Systems)

Outline

- Memories in general
 - Characteristics, types, operations, interfaces, timing diagrams, composition of memories
 - Memory performance parameters
- Read Only Memories (ROM)
 - Mask Programmable ROMs, OTP-ROMs
 - EPROM, E²PROM, NOR Flash, NAND Flash
 - FAMOS and FLOTOX transistors
- Random Access Memories (RAM)
 - Static RAMs
 - Dynamic RAMs, SDRAM, DDR
- Cache Memories
 - CAM, Cache
- Memory controllers

Introduction

- Computing systems need large, fast and cheap memories
 - DRAM: large, cheap but not fast
 - SRAM: fast, but not large and cheap
 - If SRAM is small, it becomes cheap
- Build a hierarchy of different memories to get high performance at low cost
 - Expensive memories → small
 - Cheap memories → large
 - The computing system should not be aware of the differences
 - It simply sees a large, fast and cheap memory

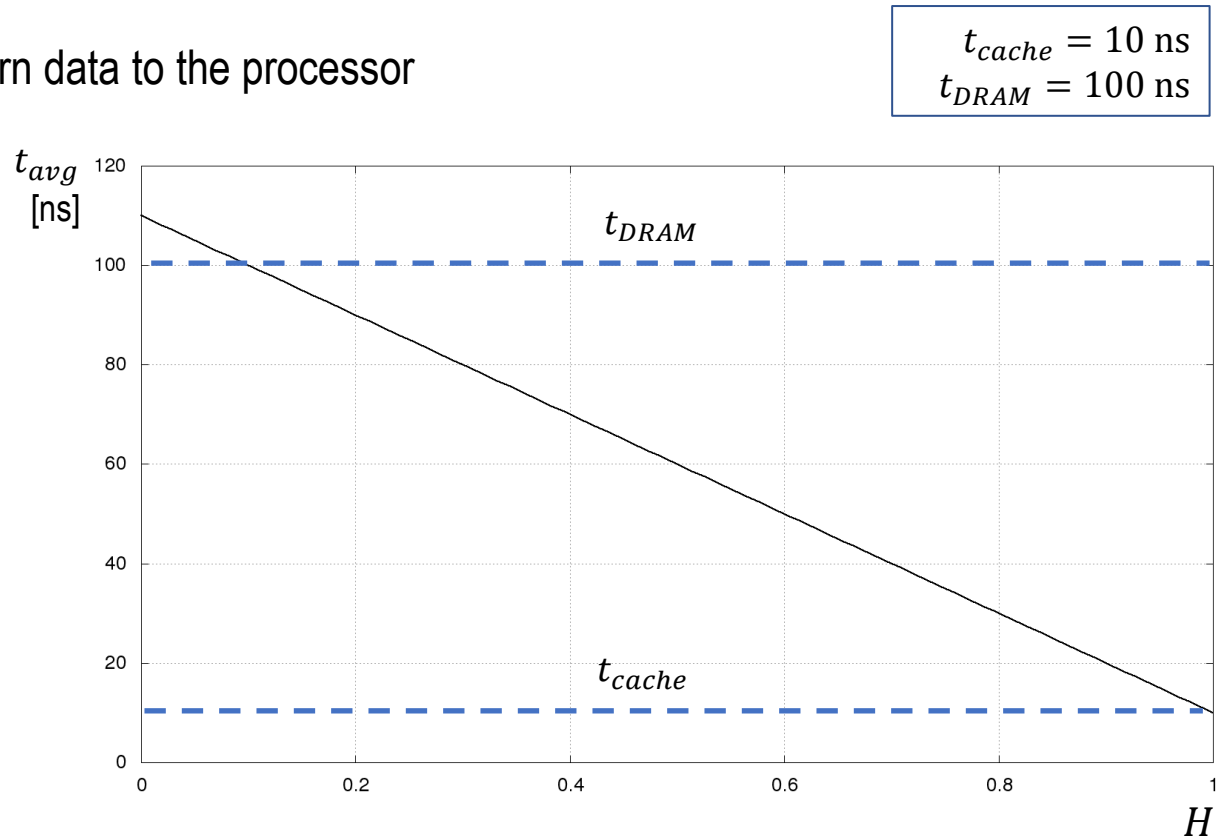


Introduction

- Cache memory
 - A FAST but SMALL static RAM (SRAM)
 - Small because SRAM are expensive
 - It virtualizes a SLOW but LARGE dynamic RAM (DRAM)
 - Large because DRAMs are cheap
 - Main memory
 - Majority of accesses are in cache
 - Substantial performance improvement with respect to a DRAM
 - Data in the large DRAM does not fit into the small SRAM
 - Keep fractions of the DRAM in cache
 - Maximize the use of the cache
 - Predict the data that is required next
 - Transfer from DRAM in cache before it is needed
 - ... while the microprocessor is doing something else in parallel

Introduction

- Cache memory performance
 - Processor always tries to read from cache
 - MISS (data is not present)
 - Read data from DRAM, store it in cache and return data to the processor
 - $t_{MISS} = t_{cache} + t_{DRAM}$
 - HIT (data is present)
 - Read directly from cache
 - $t_{HIT} = t_{cache}$
 - Average time depends on the HIT Rate H
 - $H = \frac{n_{HIT}}{n_{TOT}}, M = \frac{n_{MISS}}{n_{TOT}} = \frac{n_{TOT} - n_{HIT}}{n_{TOT}} = 1 - H$
 - $t_{avg} = Ht_{HIT} + (1 - H)t_{MISS}$
 - $t_{avg} = Ht_{cache} + (1 - H)(t_{cache} + t_{DRAM})$
 - For best performance
 - Need to maximize H



Introduction

- Cache memory performance
 - How to improve the HIT Rate
 - Data accessed more often kept in cache
 - Exploit TEMPORAL locality of programs
 - If you access some memory location, it is likely that you will access the same location again in the near future
 - Once some main memory content is in cache, don't remove it unless it is necessary
 - Examples
 - Repeatedly read parameters to process some incoming data
 - Execute a loop
 - Instructions are coded and stored in memory
 - Instructions are fetched from memory when necessary
 - First iteration of the loop → All fetches are MISSES
 - All other iterations → Fetches might be all HITS

```
1  #define ASIZE 100
2  int a[ASIZE];
3  ...
4  // Compute average of array a
5  int sum = 0;
6  for (int i = 0; i < ASIZE; i++) {
7      sum += a[i];
8  }
9  int avg = sum / ASIZE;
10
```

C code

```
4000:  mov 0, r0
4004:  mov 0, r1
4008:  mov <a>, r2
400C:  jmp loopend
      loopstart:
4010:  mov r2 + r1 * 4, r3
4014:  add r3, r0
4018:  add 1, r1
      loopend:
401C:  cmp 99, r1
4020:  jle loopstart
4024:  ...
```

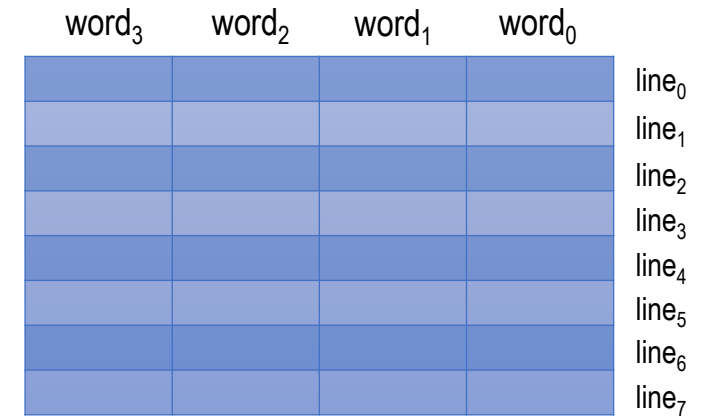
Assembly code

$r0 \leftarrow \text{sum}$
 $r1 \leftarrow i$
 $r2 \leftarrow \text{start address of array a}$
 $r3 \leftarrow \text{temp register}$

Each instruction, opcode + operand, occupies 4 bytes in memory

Introduction

- Cache memory performance
 - How to improve the HIT Rate
 - A cache does not usually store individual words
 - Multiple contiguous words → cache line
 - Every time a word is needed that is not in cache
 - An entire line is read from DRAM and stored in SRAM
 - Reading several contiguous words in DRAM allows to exploit fast access modes
 - Fast Page Mode, EDO Mode, Burst in SDRAM and DDR memories
 - Exploit SPATIAL locality of programs
 - If you access some data, it is likely that you will access other data close to it in the near future
 - Example
 - Same loop as before
 - Adjacent instructions are loaded in cache before they are fetched
 - Data of the array $a[]$ is partly loaded before it is needed



Cache Memory SRAM

```
4000:    mov 0, r0
4004:    mov 0, r1
4008:    mov <a>, r2
400C:    jmp loopend
        loopstart:
4010:    mov r2 + r1 * 4, r3
4014:    add r3, r0
4018:    add 1, r1
        loopend:
401C:    cmp 99, r1
4020:    jle loopstart
4024:    ...
```

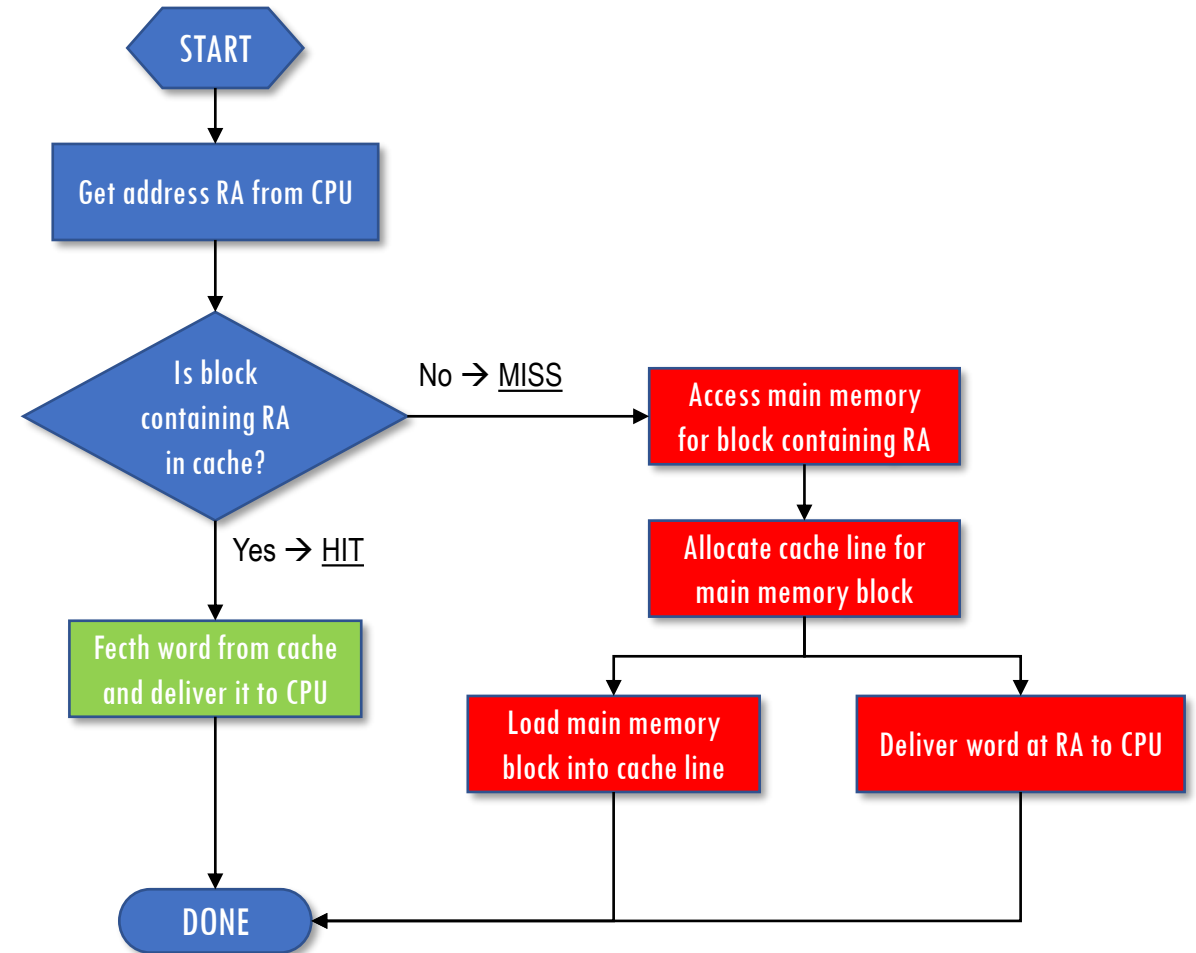
Assembly code

Introduction

- Cache memory performance and costs
 - Performance
 - A large cache potentially makes execution faster
 - More data fits in the cache, the HIT Rate is higher
 - Once all code and data is in cache → no improvement in making the cache even larger
 - Large memories are slower
 - Higher parasitic capacitances, higher t_{cache}
 - Checking cache for data takes time
 - Large memories consume more power
 - Costs
 - A large cache is expensive
 - Cost is additional to the large DRAM
 - Trade-off between costs and performance
 - It is often application dependent

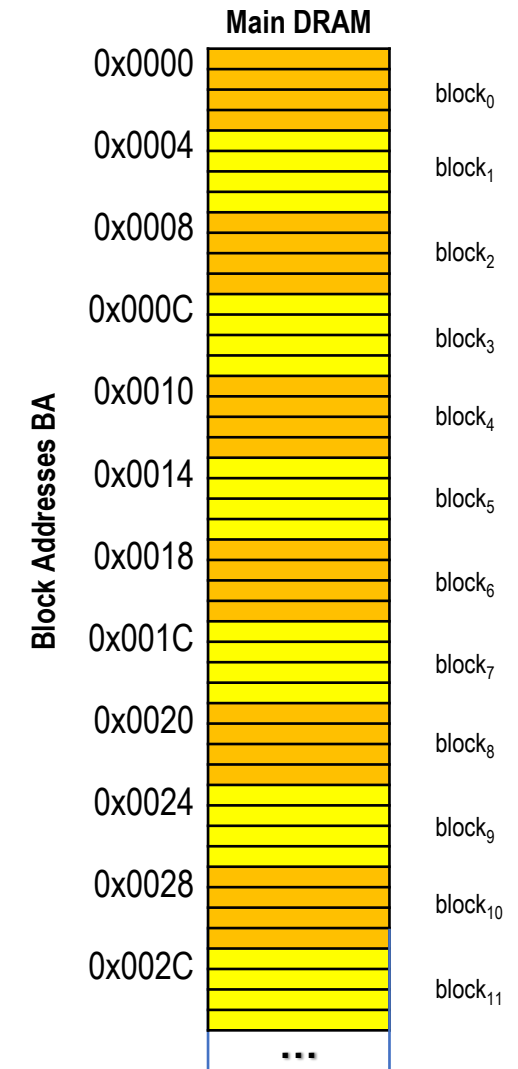
Introduction

- Cache memory policies
 - Data in DRAM is divided in blocks
 - A block has the same size of a cache line
 - Blocks are aligned to the cache line size
 - Data is always moved to and from the cache in blocks
 - The microprocessor generates a main memory address
 - RA: Read Address
 - The word at RA belongs to a certain block
 - Addresses in DRAM and cache are different
 - Need to find a fast way of knowing whether a given main memory block is already stored in cache or not



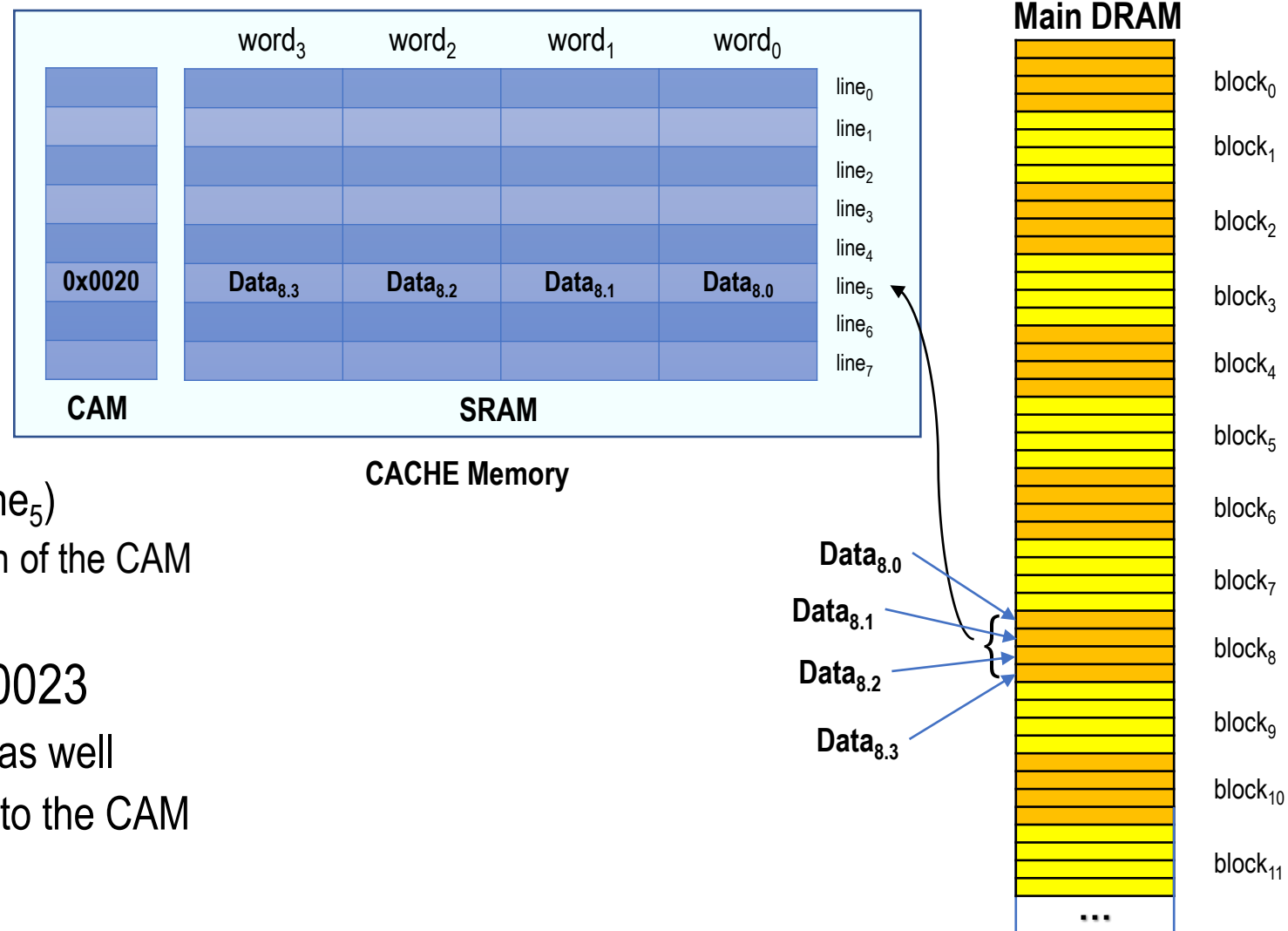
Introduction

- How do we know if the required data is already in cache?
 - Data is identified by its address RA in main memory
 - Since the memory is always moved in blocks, the block address BA is sufficient
 - Use a CAM to store block addresses in cache
 - The CAM is additional to the SRAM
 - The number of locations in the CAM is equal to the number of cache lines
 - For each cache line, it stores the main memory (DRAM) block address BA
 - Or part of it, just enough to identify the block
 - It returns the cache (SRAM) block address (i.e. the index of the cache line)
 - When reading
 - Compute the block address BA starting from the word address RA
 - Match the block address BA (or part of it) in the CAM
 - HIT: use the returned SRAM address to get the word
 - MISS: load the block from DRAM, store it in SRAM, store its address in the CAM, return the word



Introduction

- Example
 - Cache line is 4 words large
 - Cache initially empty
 - Read word at address RA=0x0022
 - It belongs to the ninth block (block₈)
 - The block address is BA=0x0020
 - Store block₈ in the sixth cache line (line₅)
 - Store its address in the sixth location of the CAM
 - Return word₂ from the cache line
 - Next, read word at address RA=0x0023
 - It belongs to the ninth block (block₈), as well
 - Match the block address BA=0x0020 to the CAM
 - HIT, the CAM returns the line₅
 - Read word₃ from the cache line

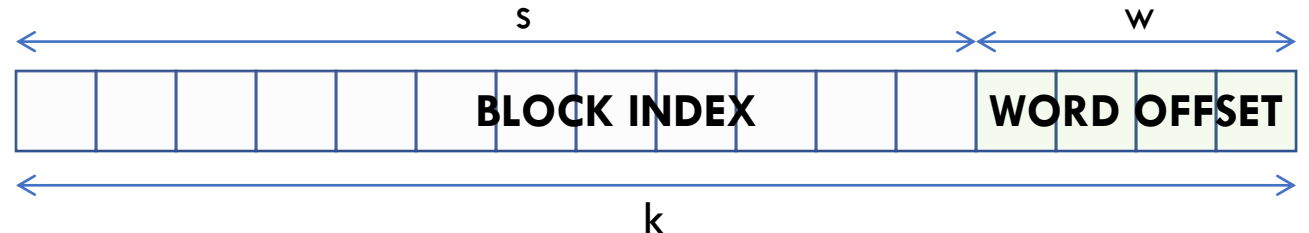


Introduction

- Cache memory policies
 - When reading from DRAM and storing in cache
 - Must decide where to write the data
 - MAPPING policy
 - If cache is full and new data has to be written
 - Some previous data must be erased
 - REPLACEMENT policy
 - When writing some data
 - Cache and main memory should be both updated
 - CONSISTENCY policy

Cache Memories

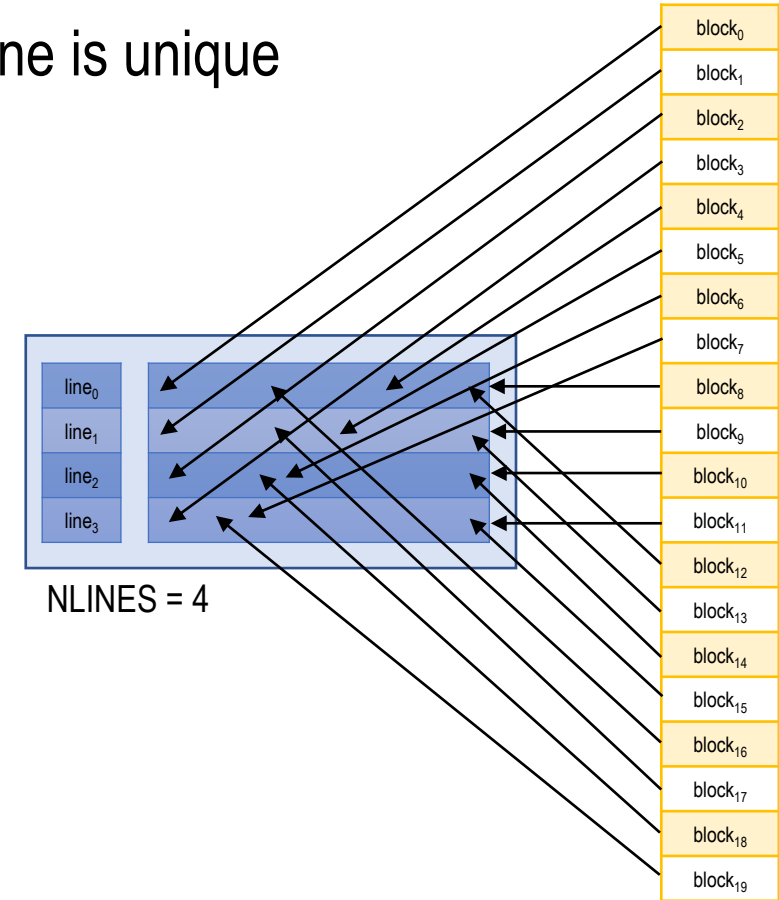
- Cache Memory Parameters
 - MAINSIZE: size of DRAM main memory (in words)
 - $k = \log_2 \text{MAINSIZE}$
 - CACHESIZE: size of SRAM (in words)
 - LINESIZE: number of words in a cache line
 - $w = \log_2 \text{LINESIZE}$
 - NLINES: number of cache lines
 - $\text{NLINES} = \text{CACHESIZE} / \text{LINESIZE}$
 - $r = \log_2 \text{NLINES}$
 - NBLOCKS: number of blocks in DRAM
 - $\text{NBLOCKS} = \text{MAINSIZE} / \text{LINESIZE}$
 - $s = \log_2 \text{NBLOCKS}$
 - $k = s + w$



Cache Memories Mapping Policies

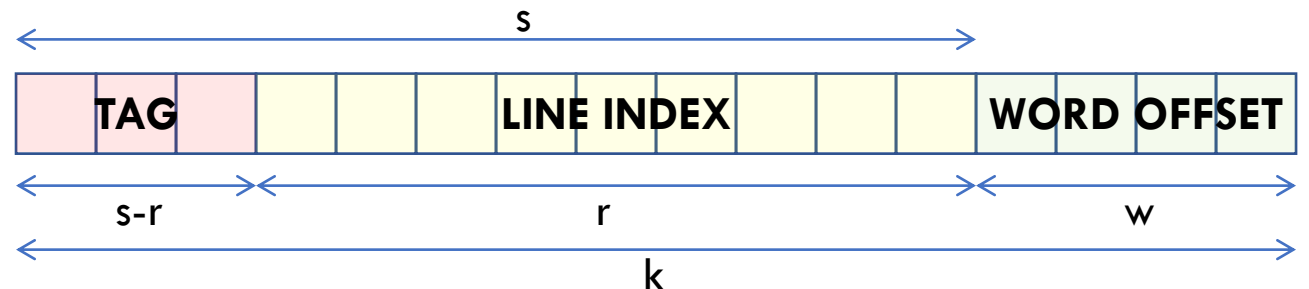
- Direct Mapping

- The association between a block in main memory and a cache line is unique
 - The 1st block (block_0) in the first cache line (line_0)
 - The 2nd block (block_1) in the second cache line (line_1)
 - ...
 - The $\text{NLINES}^{\text{th}}$ block ($\text{block}_{\text{NLINES}-1}$) in the last cache line ($\text{line}_{\text{NLINES}-1}$)
 - The $(\text{NLINES}+1)^{\text{th}}$ block ($\text{block}_{\text{NLINES}}$) in the first cache line (line_0)
 - ...
 - The $(2\text{NLINES})^{\text{th}}$ block ($\text{block}_{2\text{NLINES}-1}$) in the last cache line ($\text{line}_{\text{NLINES}-1}$)
 - The $(2\text{NLINES}+1)^{\text{th}}$ block ($\text{block}_{2\text{NLINES}}$) in the first cache line (line_0)
 - ...
- Multiple blocks assigned to the same cache line
 - There are $\text{NBLOCKS}/\text{NLINES}$ blocks mapped to each cache line
 - Only one of them is loaded in cache at any given time



Cache Memories Mapping Policies

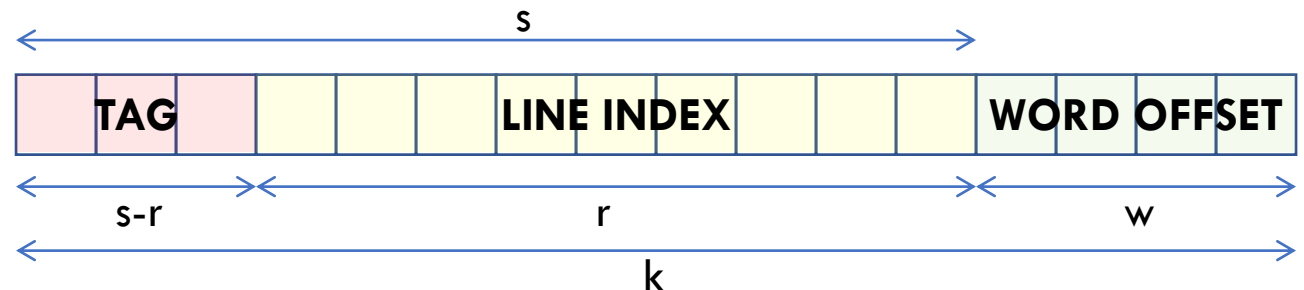
- Direct Mapping
 - How to quickly compute the LINE INDEX of a block
 - Easy because all parameters are power of 2
 - From the Word ADDRESS, compute the BLOCK number and its address in memory
 - $\text{BLOCKINDEX} = \text{ADDRESS} \gg w$ (on s bits)
 - $\text{BLOCKADDRESS} = \text{ADDRESS} \& (\sim((1 \ll w) - 1))$
 - On k bits, it is the Word ADDRESS with the lowest w significant bits at 0
 - From the BLOCKINDEX, compute the LINE INDEX
 - $\text{LINE} = \text{BLOCKINDEX} \% \text{NLINES}$ (on r bits)
 - In practice, no need to compute anything
 - Can be directly obtained from the Word ADDRESS by proper masking and/or shifting



Cache Memories Mapping Policies

- Direct Mapping

- Consider blocks in main memory mapped to the same cache line
 - The address of each block always has the w least significant bits at 0
 - Do not need to store this information in the CAM
 - The LINE INDEX is the same for all blocks assigned to the same cache line
 - It corresponds to the cache line to which they are mapped
 - Do not need to store this information in the CAM
 - The TAGs differ
 - TAG is made of the upper $(s - r)$ bits of the address
 - This is the only information that needs to be stored in the CAM
 - Different blocks mapped to the same cache line are guaranteed to have different TAGs



Cache Memories Mapping Policies

- Direct Mapping

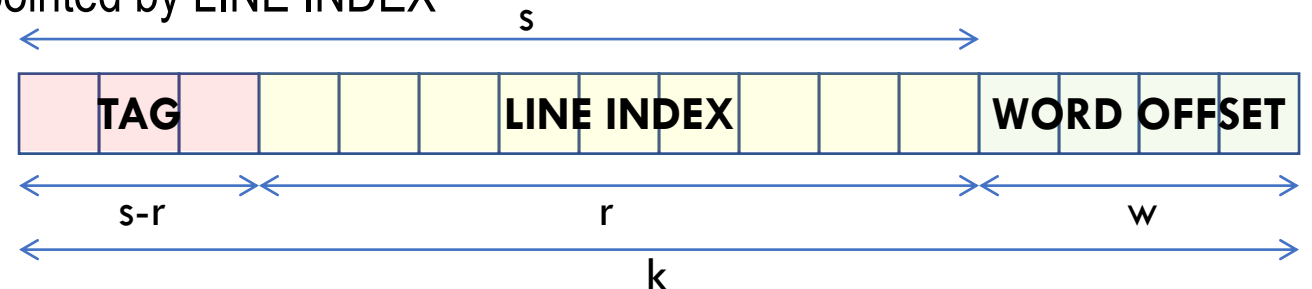
- Given the address of some data
 - Match the TAG with the CAM at location pointed by LINE INDEX (only if entry is valid)
 - The CAM is actually made of NLINES separate CAMs of a single word

- If MISS

- Read the memory block from DRAM at address BLOCKADDRESS
- TAG is stored in the CAM, in the location pointed by LINE INDEX
- The block data is stored in the SRAM, in the line pointed by LINE INDEX
 - Possibly replacing other data already present
- VALID bit is set for the line
- Read word at WORD OFFSET in the line pointed by LINE INDEX

- If HIT

- Read word at WORD OFFSET in the line pointed by LINE INDEX

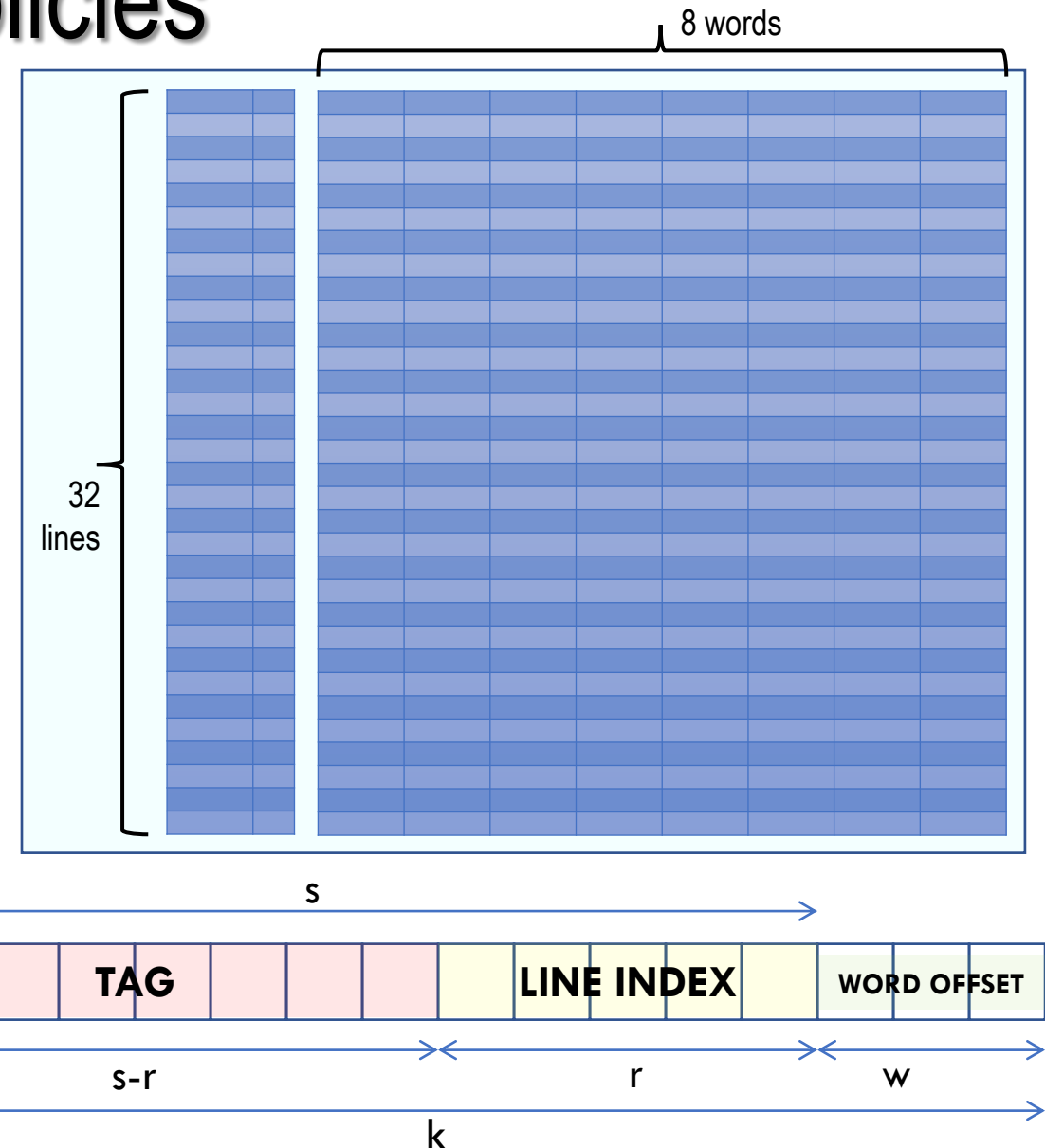


Cache Memories Mapping Policies

- Direct Mapping

- Example

- MAINSIZE = 64 Kwords (word = 8 bits)
 - $k = 16$
 - LINESIZE = 8 words
 - $w = 3$
 - CACHESIZE = 256 bytes
 - $NLINES = CACHESIZE / LINESIZE = 32$
 - $r = 5$
 - $NBLOCKS = MAINSIZE / LINESIZE = 8192$
 - $s = 13$
 - TAG
 - $s - r = 8$

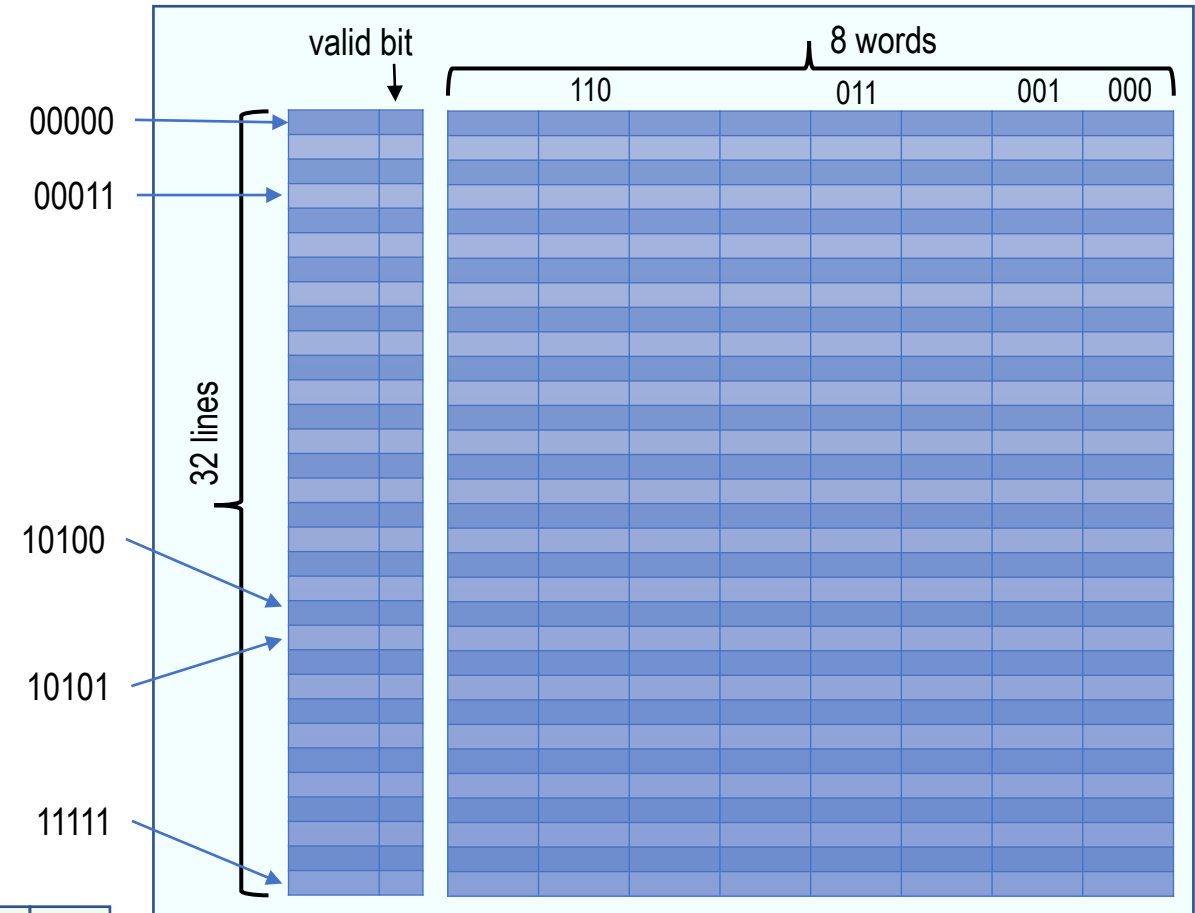
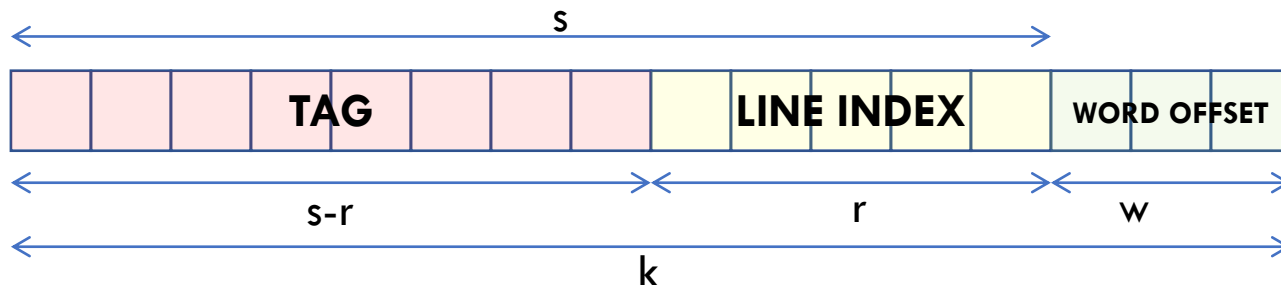


Cache Memories Mapping Policies

- Direct Mapping

- Example

- Address 0x34A1 (0011010010100001)
 - Address 0x34A6 (0011010010100110)
 - Address 0x34A0 (0011010010100000)
 - Address 0x34A8 (0011010010101000)
 - Address 0x28A3 (0010100010100011)
 - Address 0x34A6 (0011010010100110)
 - Address 0x851B (1000010100011011)

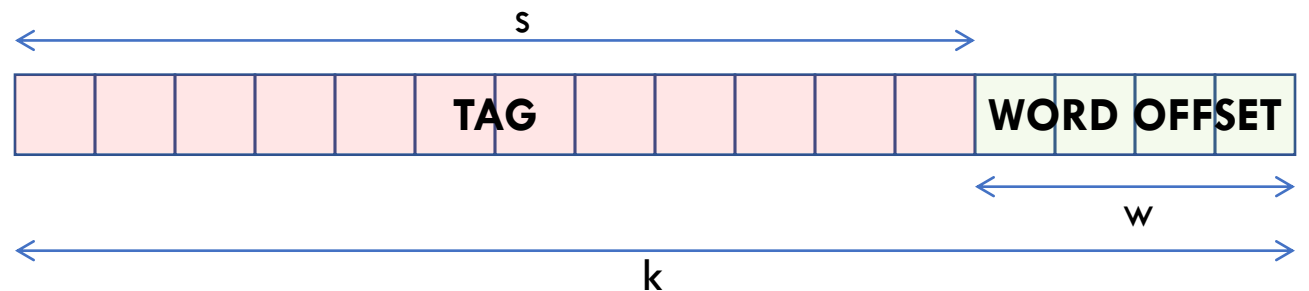


Cache Memories Mapping Policies

- Direct Mapping
 - Advantages
 - Very simple to implement, cheap
 - Don't even need a CAM!
 - LINE INDEX already points to the only location to match
 - Low power consumption
 - Simple replacement policy
 - No policy at all
 - Disadvantages
 - May replace some data even if cache is not full
 - Poor performance
 - The HIT Rate is the poorest

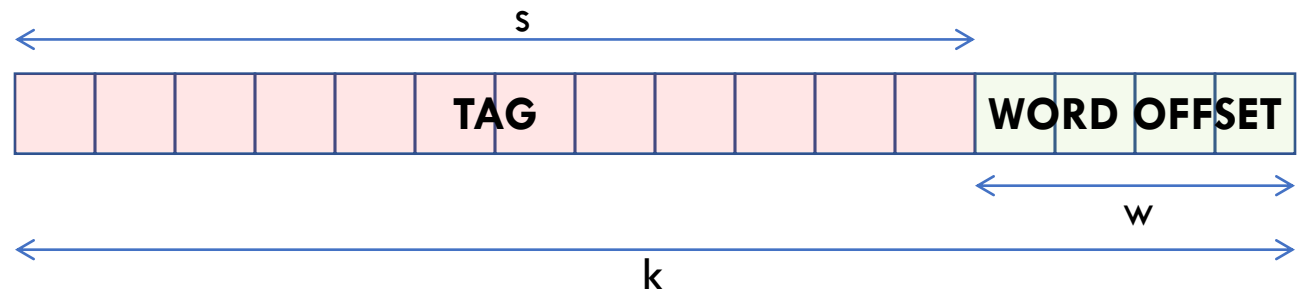
Cache Memories Mapping Policies

- Fully Associative Mapping
 - No pre-determined association between a main memory block and a line
 - When loaded from DRAM, a block is stored in the first available cache line
 - There is no pre-determined LINE INDEX
 - LINE INDEX is 0 bit long
 - The address of each block always has the w least significant bits at 0
 - These bits correspond to the WORD OFFSET
 - Do not need to store this information in the CAM
 - TAG is all the remaining bits of the ADDRESS
 - TAG is s bit long
 - TAG is identical to BLOCKINDEX



Cache Memories Mapping Policies

- Fully Associative Mapping
 - Given the address of some data
 - Match the TAG with the entire CAM (only valid entries)
 - If MISS
 - Read the memory block from DRAM at address BLOCKADDRESS
 - TAG is stored in the CAM, in the next empty line
 - Need a replacement policy if cache is full
 - The block data is stored in the SRAM, in the same line as TAG
 - VALID bit is set for the line
 - Read word at WORD OFFSET in the line
 - If HIT
 - Read word at WORD OFFSET in the line that generated HIT

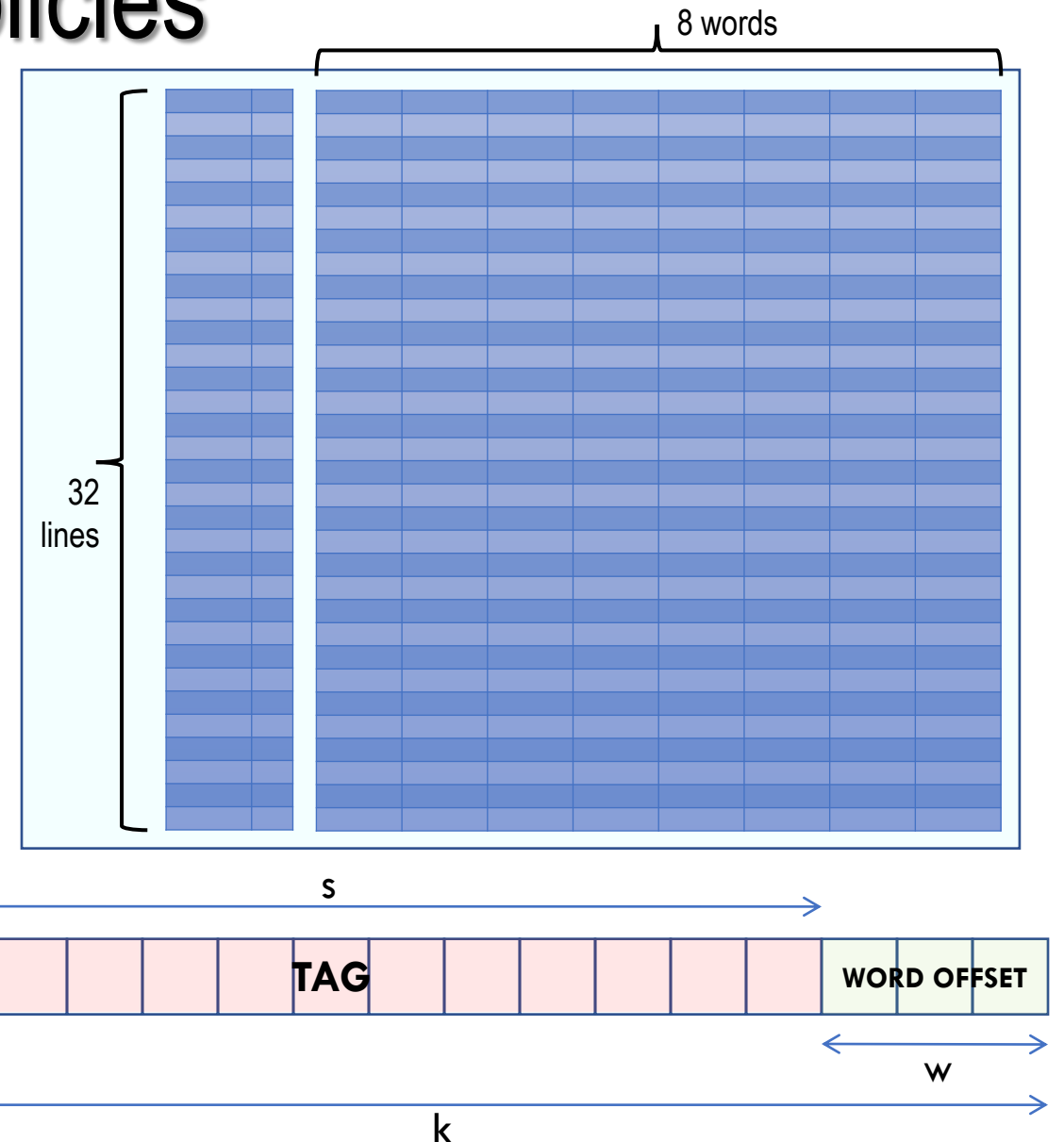


Cache Memories Mapping Policies

- Fully Associative Mapping

- Example

- MAINSIZE = 64 Kwords (word = 8 bits)
 - $k = 16$
 - LINESIZE = 8 words
 - $w = 3$
 - CACHESIZE = 256 bytes
 - NLINES = CACHESIZE / LINESIZE = 32
 - $r = 5$ (but LINE INDEX not used)
 - NBLOCKS = MAINSIZE / LINESIZE = 8192
 - $s = 13$
 - TAG
 - $s - 0 = 13$

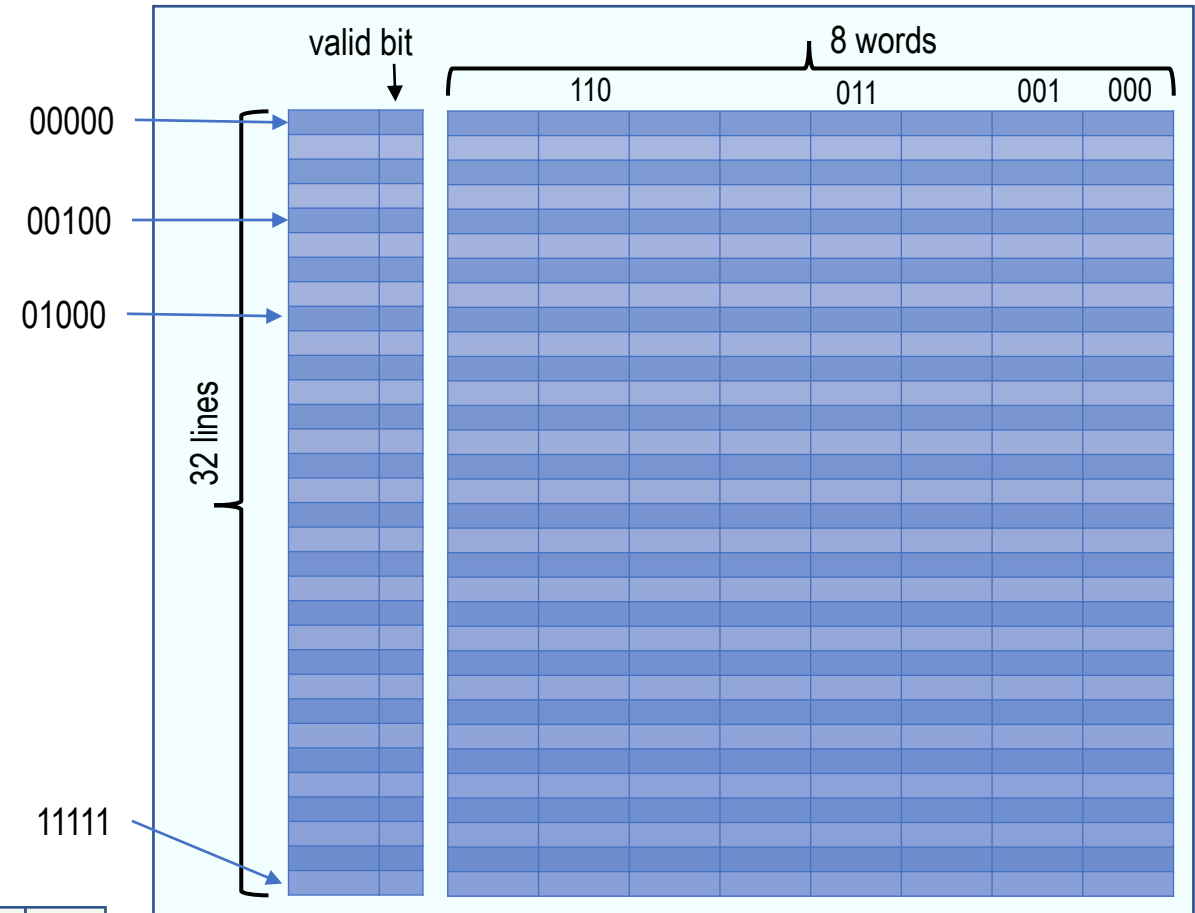
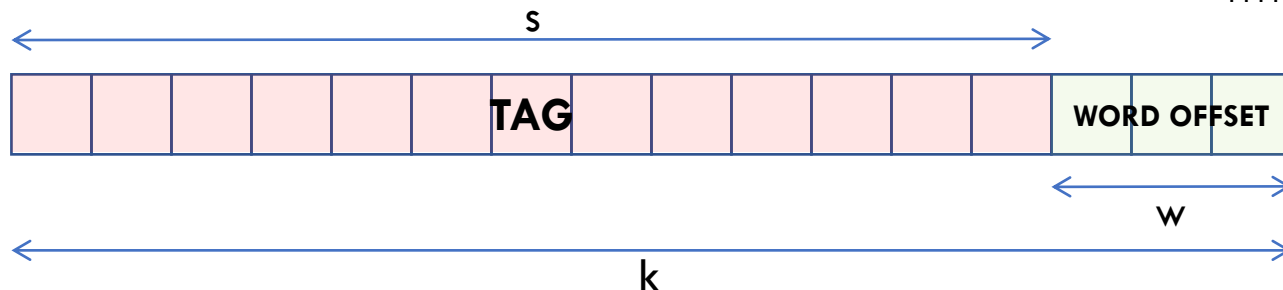


Cache Memories Mapping Policies

- Fully Associative Mapping

- Example

- Address 0x34A1 (0011010010100001)
- Address 0x34A6 (0011010010100110)
- Address 0x34A0 (0011010010100000)
- Address 0x34A8 (0011010010101000)
- Address 0x28A3 (0010100010100011)
- Address 0x34A6 (0011010010100110)
- Address 0x851B (1000010100011011)



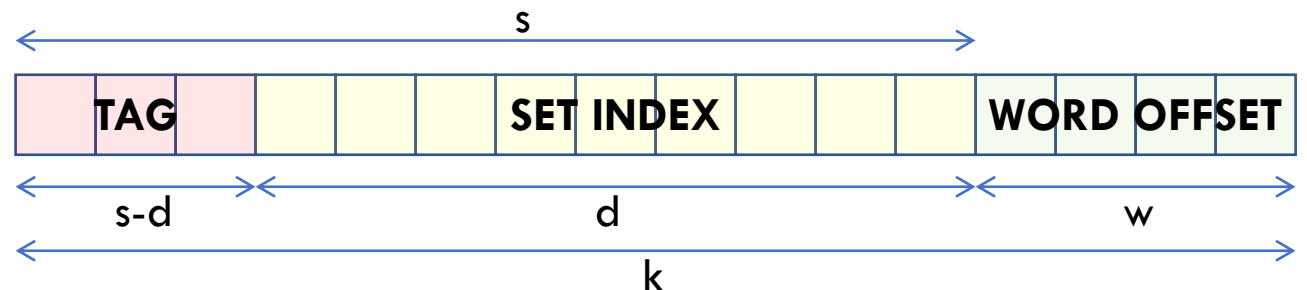
Cache Memories Mapping Policies

- Fully Associative Mapping
 - Advantages
 - Best use of the cache
 - Best hit rate
 - Possibly best performance
 - Disadvantages
 - Complex to implement
 - Expensive
 - Need a (large) CAM
 - Power dissipation
 - Higher delays

Cache Memories Mapping Policies

- Set Associative Mapping

- Cache lines are grouped into sets
 - NSETS: number of sets
 - SETLINES: number of lines in a set
 - $NSETS = NLINES / SETLINES$
 - Bits to identify a set
 - $d = \log_2(NSETS)$
- Direct association of a main memory block towards a set
- Fully Associative association within that set
- From the BLOCKINDEX, compute the SET index
 - $SET = BLOCKINDEX \% NSETS$ (on d bits)
 - Can be directly obtained from the Word ADDRESS
- TAG is $(s - d)$ bits long



Cache Memories Mapping Policies

- Set Associative Mapping

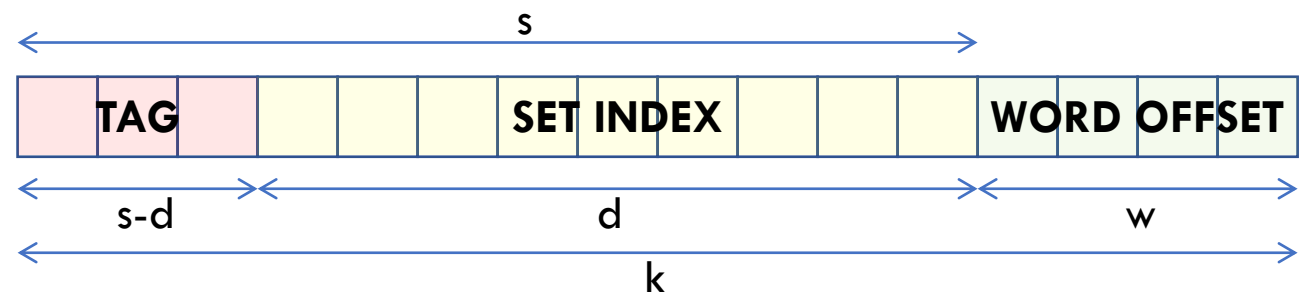
- Given the address of some data
 - Match the TAG with the CAM of the SET pointed by SET INDEX (only valid entries)
 - There are NSETS separate CAMs, each of SETLINES entries

- If MISS

- Read the memory block from DRAM at address BLOCKADDRESS
- TAG is stored in the CAM of the SET, in the next empty line
 - Need a replacement policy if SET is full
- The block data is stored in the SRAM, in the same line as TAG
- VALID bit is set for the line
- Read word at WORD OFFSET in the line

- If HIT

- Read word at WORD OFFSET in the line that generated HIT

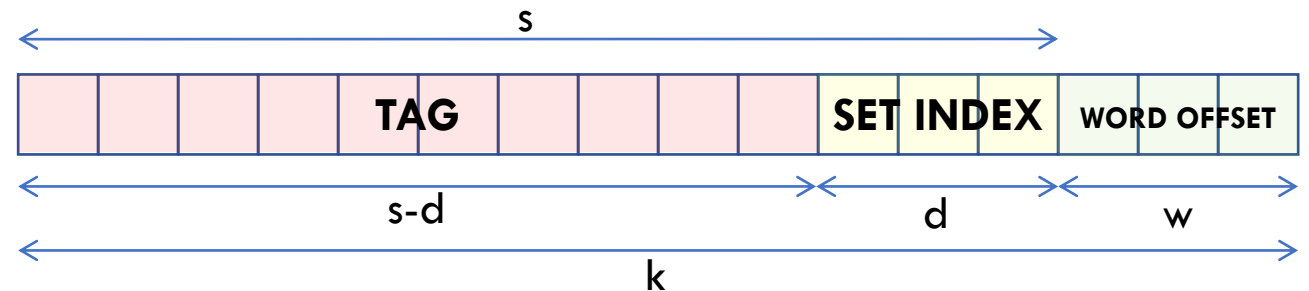
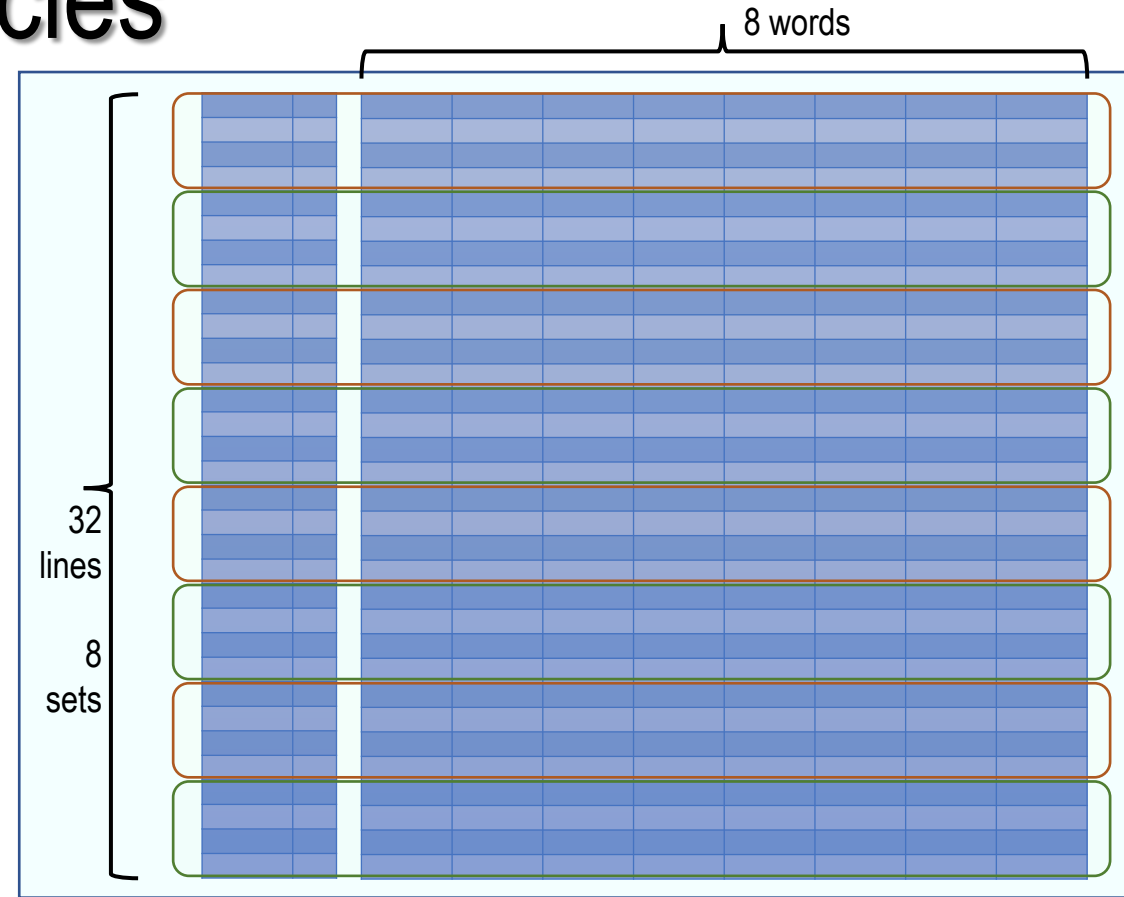


Cache Memories Mapping Policies

- Set Associative Mapping

- Example

- MAINSIZE = 64 Kwords (word = 8 bits), $k = 16$
 - LINESIZE = 8 words, $w = 3$
 - CACHESIZE = 256 bytes
 - NLINES = CACHESIZE / LINESIZE = 32
 - $r = 5$ (but LINE INDEX not used)
 - NSETS = 8, SETLINES = 4, $d = 3$
 - NBLOCKS = MAINSIZE / LINESIZE = 8192
 - $s = 13$
 - TAG
 - $s - d = 13 - 3 = 10$

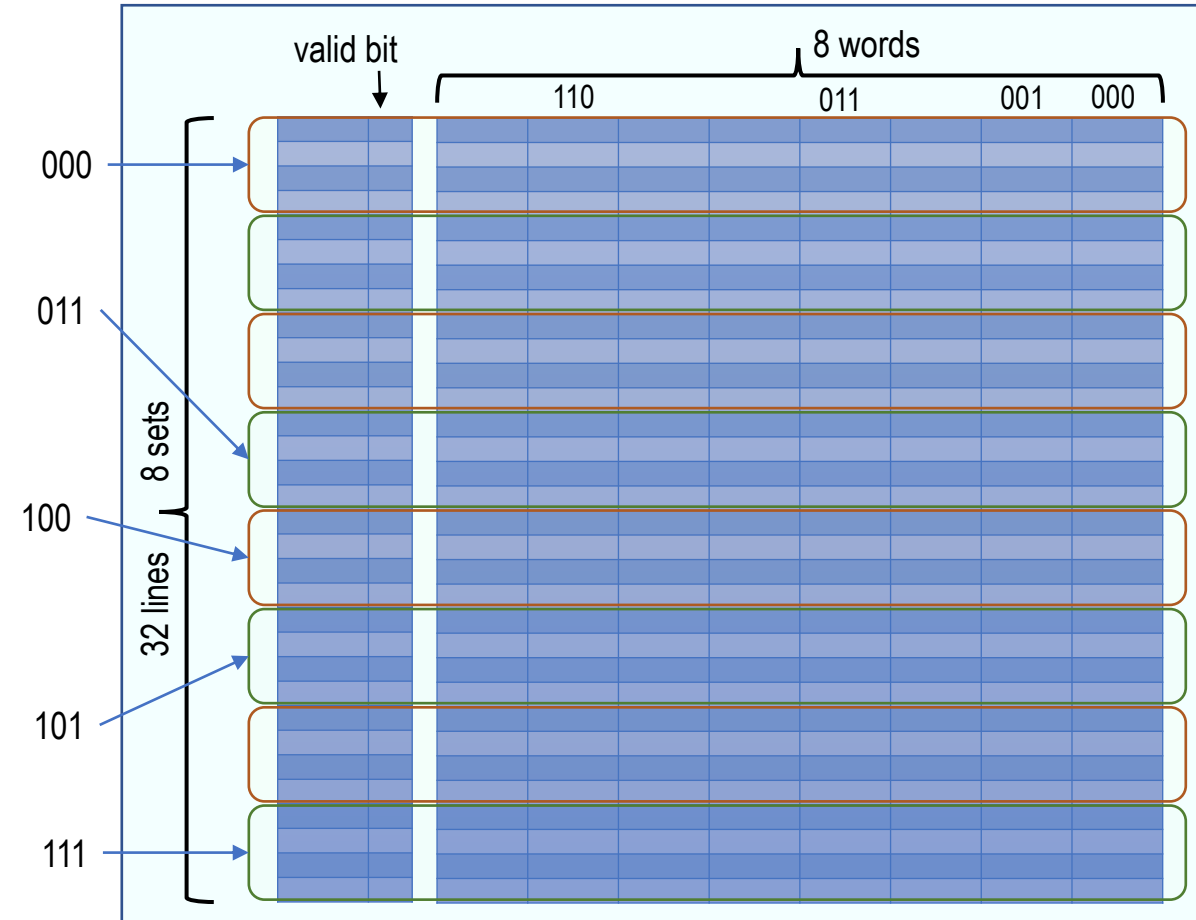
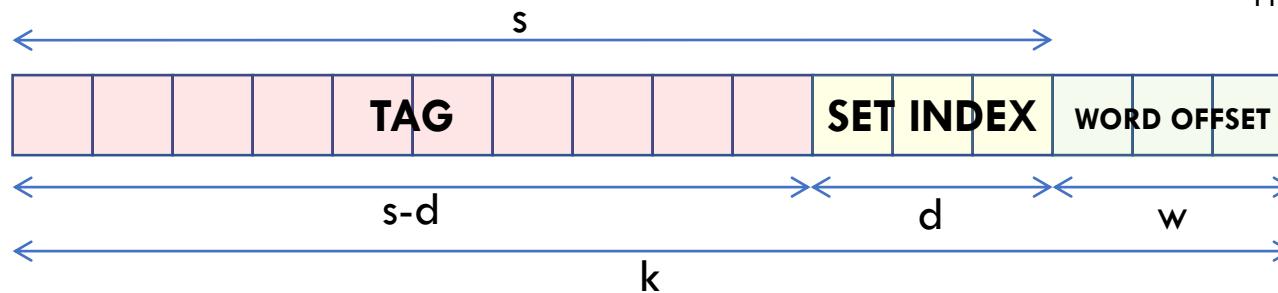


Cache Memories Mapping Policies

- Set Associative Mapping

- Example

- Address 0x34A1 (0011010010100001)
- Address 0x34A6 (0011010010100110)
- Address 0x34A0 (0011010010100000)
- Address 0x34A8 (0011010010101000)
- Address 0x28A3 (0010100010100011)
- Address 0x34A6 (0011010010100110)
- Address 0x851B (1000010100011011)



Cache Memories Mapping Policies

- Set Associative Mapping
 - Advantages
 - Good use of the cache
 - Good performance
 - Can find a trade-off between complexity, performance and power consumption
 - Disadvantages
 - Need several small CAMs
 - 2-way set associative
 - 2 lines in each set
 - N-way set associative
 - N lines in each set

Cache Memories Mapping Policies

- Set Associative Mapping is a general case

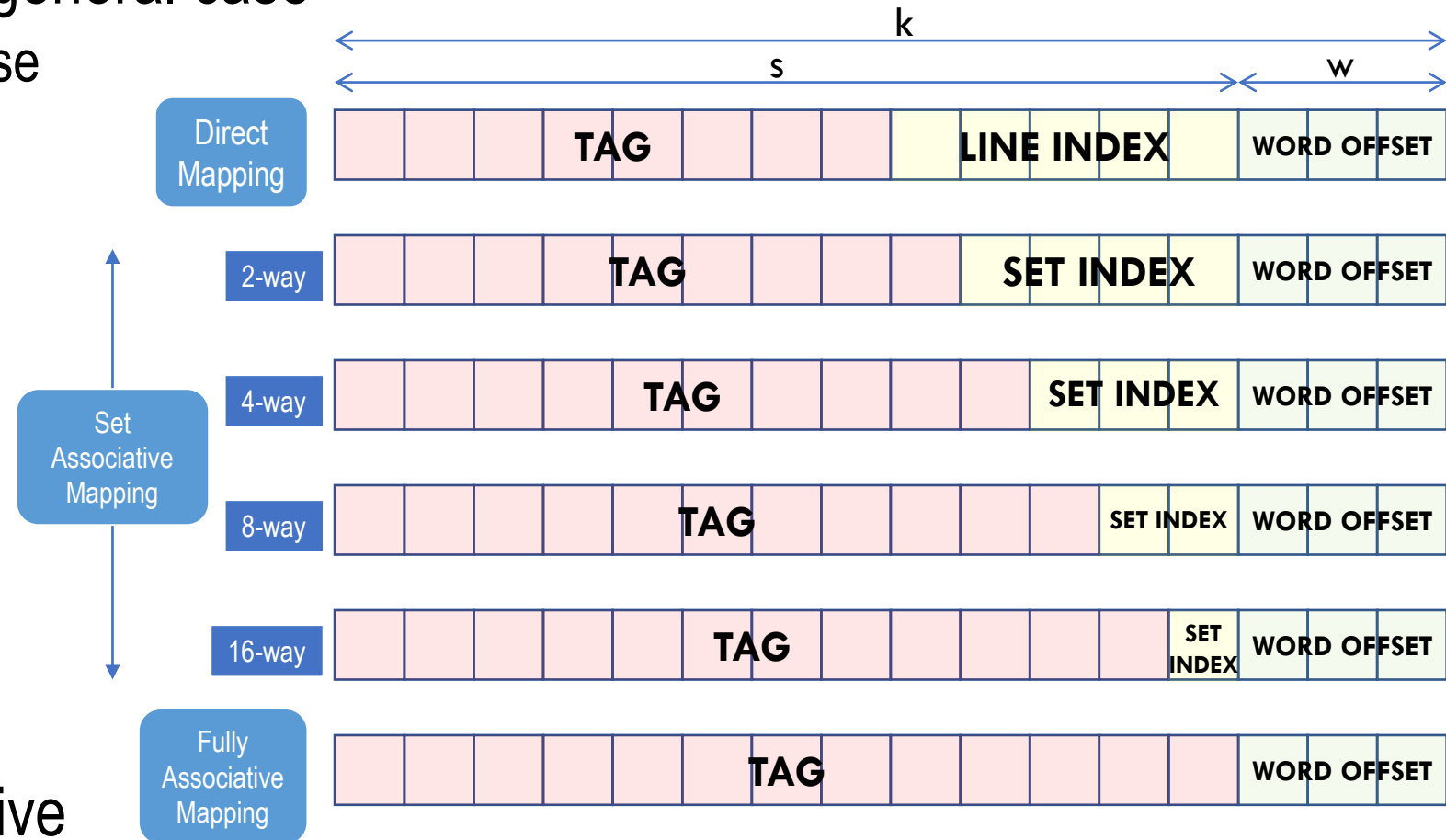
- Direct mapping is a special case of set associative mapping

- $SETLINES = 1$
- $NSETS = NLINES$
- $d = r$

- Fully associative mapping is a special case of set associative mapping

- $NSETS = 1$
- $SETLINES = NLINES$
- $d = r = 0$

- Most caches are Set Associative



Cache Memories Replacement Policies

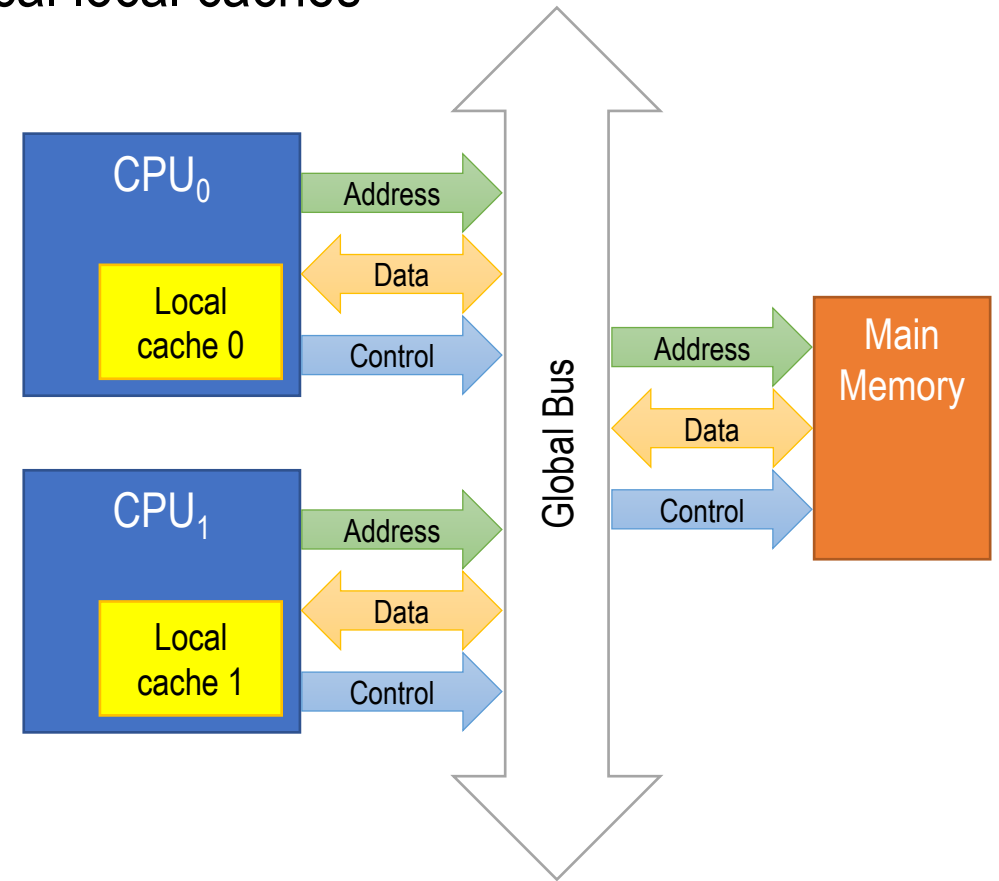
- When a cache is full, a cache line must be replaced to store a new one
 - Direct Mapping
 - No choice, may replace a cache line even if the cache is not full
 - Associative Mapping
 - Set Associative Mapping may replace a cache line even if the cache is not full (but the target set is full)
 - Least Recently Used (LRU)
 - Store timestamp of last usage
 - Need to sort timestamps (within a set or within the full cache, for Set and Fully associative, respectively)
 - Least Frequently Used (LFU)
 - Use counters to know the number of accesses
 - Need to sort number of accesses (within a set or within the full cache, for Set and Fully associative, respectively)
 - First In First Out (FIFO)
 - Restart from first line when the bottom (of a set or the full cache) is reached
 - A simple pointer is required (one for each set, or one for the full cache, for Set and Fully associative, respectively)
 - Random

Cache Memories Consistency Policies

- When writing, how to update cache and main memory
 - Important especially in multi-core systems and with peripherals accessing the memory
 - Each core has a dedicated cache
- Write through
 - All writes go to main memory as well as cache
 - Multiple CPUs can monitor main memory traffic to keep local caches up to date
 - When a core makes a write, the other cores invalidate the cache entry relative to the written block
 - Lots of traffic
 - Slows down write
 - Best consistency
 - On average, only 15% of memory references are writes

Cache Memories Consistency Policies

- Write through example
 - Assume for simplicity that CPU₀ and CPU₁ have identical local caches
 - CPU₀ reads data at address 0x0CF780
 - Stores an entire block in cache
 - CPU₁ reads data at address 0x0CF782
 - Stores the same entire block in cache
 - CPU₀ reads data at address 0x0CF781
 - It is a cache HIT, nothing happens on the global bus
 - CPU₀ writes data at address 0x0CF781
 - It updates both the cache and the main memory
 - The write appears on the Global Bus
 - CPU₁ detects the write at address 0x0CF781
 - It invalidates the cache entry for that block

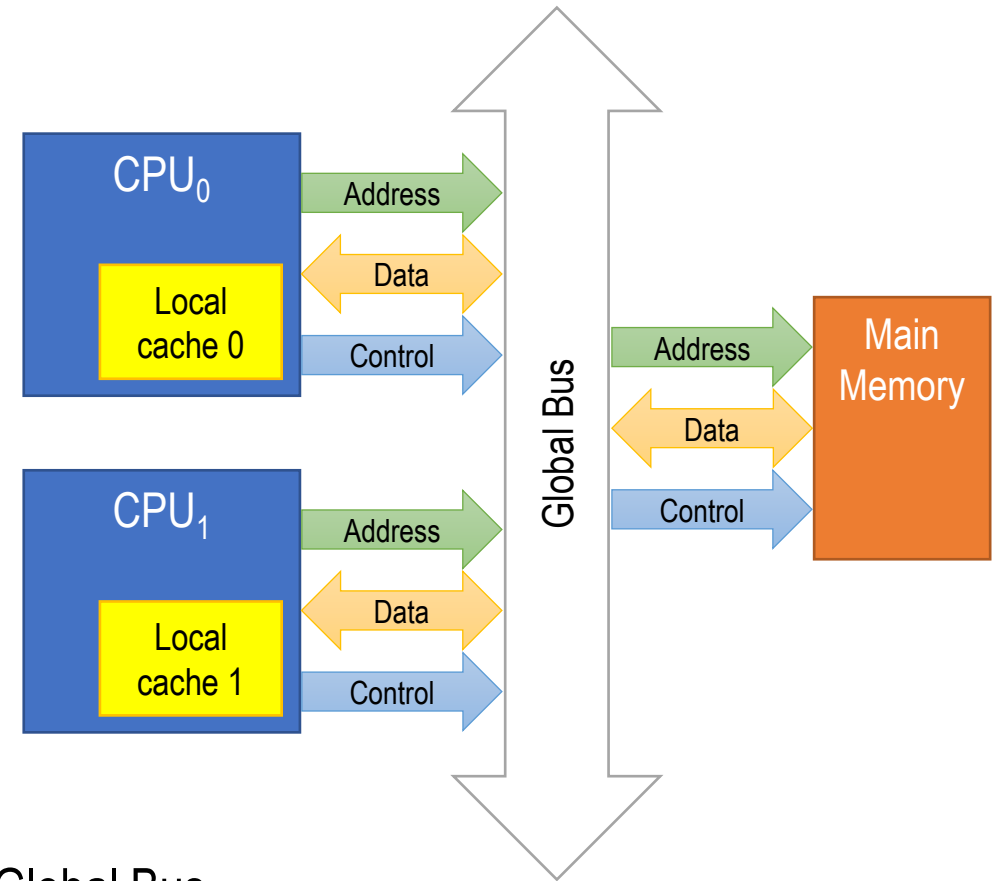


Cache Memories Consistency Policies

- When writing, how to update cache and main memory
 - Important especially in multi-core systems and with peripherals accessing the memory
 - Each core has a dedicated cache
- Write back
 - Update initially made in cache only
 - Update bit is set for cache line
 - Called *dirty* bit
 - If line is to be replaced in cache
 - Write to main memory if dirty bit is set
 - Saves a lot of traffic on the memory bus
 - Improved performance
 - Other caches get out of sync

Cache Memories Consistency Policies

- Write back example
 - CPU₀ reads data at address 0x0CF780
 - Stores an entire block in cache
 - CPU₁ reads data at address 0x0CF782
 - Stores the same entire block in cache
 - CPU₀ writes data at address 0x0CF781
 - It updates only the cache, NOT the main memory
 - It sets the dirty bit for the cache line
 - The write does not appear on the Global Bus
 - CPU₁ reads data at address 0x0CF781
 - It is a cache HIT, nothing happens on the global bus
 - But the data read is old
 - CPU₀ needs to replace the cache entry
 - The block is written back to main memory, appearing on the Global Bus

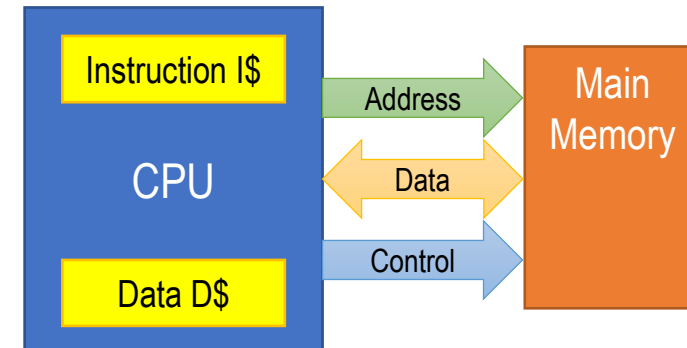


Cache Memories

- Example of performance
- Larger cache achieves lower miss rate but higher access cost
 - 2 Kbyte cache, miss rate = 15%, hit cost = 2 cycles, miss cost = 20 cycles
 - $t_{avg} = (0.85 \cdot 2) + (0.15 \cdot 20) = 4.7$ cycles
 - 4 Kbytes cache, miss rate = 6.5%, hit cost = 3 cycles, miss cost = 20 cycles
 - $t_{avg} = (0.935 \cdot 3) + (0.065 \cdot 20) = 4.105$ cycles
 - 8 Kbytes cache, miss rate = 5.565%, hit cost = 4 cycles, miss cost = 20 cycles
 - $t_{avg} = (0.94435 \cdot 4) + (0.05565 \cdot 20) = 4.8904$ cycles
 - The best cache is not necessarily the largest cache
- Improve cache hit rate without increasing size
 - Increase line size
 - Change set-associativity

Cache Memories

- Example of multiple caches
 - Modified Harward architecture
 - Separate Instruction and Data caches
 - May have different size, word size, associativity
 - Common external address space on main memory
 - Processor can fetch instruction while concurrently accessing data
 - Improved performance
 - A single main memory is simple
 - Better use of available resources
 - Reduced costs

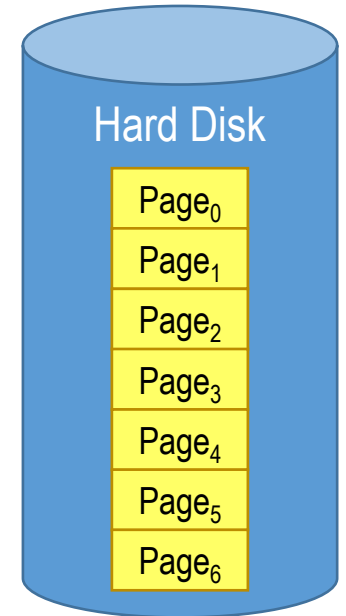
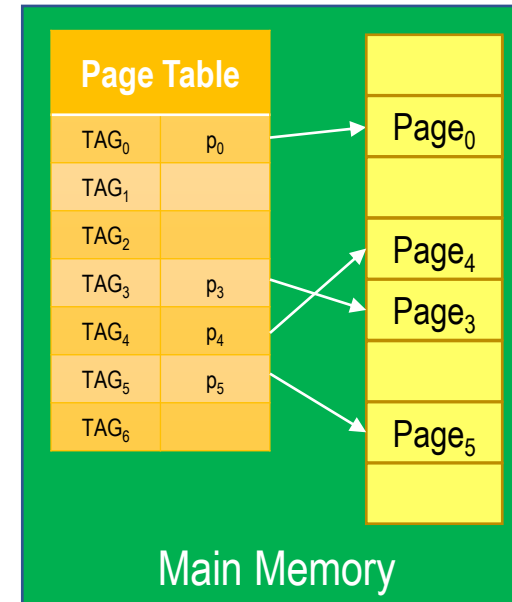


Virtual Memory

- Processor address space is larger than physical memory
 - Virtualize part of the memory on disk
- Different processes (or threads) see separate address spaces
 - Each process has its own virtual address
 - Protection between processes
- Memory is divided in pages of fixed size
 - A page can be in physical memory or on disk
- Each process has a *page* table that holds the correspondence between the virtual address of a page, and its physical address, if it exists
 - TAGs in the page table are the upper part of the virtual address (i.e. the page address)
 - Entries in the page table are the translation address, if page is in memory
 - Page tables are always in physical memory

Virtual Memory

- Two memory accesses to read or write memory
 - First one to read the page table
 - Second one to access the actual page
- If page is not in main memory
 - Page fault
 - The operating system loads the page from disk
 - Stores the page in main memory
 - Updates the page table
 - The instruction that caused the page fault is re-issued



Virtual Memory

- How to improve performance?
 - Use a cache of translations
 - *Translation Lookaside Buffer*, TLB
 - Similar to page table
 - The TAG is the upper part of the virtual address
 - The entry is the page address in physical memory
 - TLB is on-chip
 - It uses a CAM
 - It is fast
 - If HIT in TLB
 - Use the translation directly to access memory
 - If MISS
 - Read the page table
 - If page is in memory, update the TLB with new translation
 - If page is on disk, perform a page fault

